

Tutorial 1

Date

No.

1. `argv[1]` the starting address of ...
`* (argv[3]+1)` 字符串最后的 '0' 别忘了... 数数数清楚
 指向0号位 指向1号位
 eg. `char c='a';`
 2. `#include <ctype.h>` } 若c是小写字母, 返回对应的大写字母
`int toupper(int c)` } 若c不是小写字母, 返回原样
`toupper(c);`
 返回值是 'A', 但变量c
 3. 问你一个数中的0-bit有多少是从16/32位来看, 而不是从所占位来看 本身还是 'a'
 5. overflow bit: 若是nbit, $C_{n-1} \wedge C_{n-2}$, 其中 C_{n-1} 是从符号位产生的进位
 eg. $1011 \rightarrow 1010 \rightarrow 0101 \rightarrow 5 \rightarrow -5$ C_{n-2} 是进入到符号位的进位
- 负数的二补码得到正数后, 不要忘了加负号...
6. in 2's complement $-a = \sim a + 1 \therefore \sim a = -a - 1$
7. 16个bit `mask = 1 << 15;` $15 << 28 >$
 32 $1 << 31;$ $\Delta \rightarrow n-1$
 一定要注意1不是2

Tutorial 2

1. `0101 5` `1000 8` `1111 f` `0110 6` ✗
`0x` → 十六进制 `0..` → 八进制
2. $0 \sim 2^{16}-1 \Rightarrow 0 \sim 65535$ $2^{16} = 65536$ ✗
3. %u读入负数时: 不会报错, 而是把负数的二进制补码当成无符号整数解释, 因此通常得到一个很大的正数
4. 优先级 (见OP3总结)

CP3

1. `printf("%d", argv[1]);` Answer: the starting address of ...
2. `if(a'b)` → not equal else → equal
3. 一定要辨别清楚输出的数据类型
4. `h7h6h5h4h3h2h1h0` → `h+h5h+h2h1h0h6h7`
`unsigned encode(unsigned this1)`

R想要特定 ?

位置上的数据, 在这 `unsigned x03, x4, x5, x6, x7;`

些位置上 & 1, 其余位 `x03 = (this1 & 0x0000ffff) << 8;`

置上 & 0 `x4 = (this1 & 0x000f0000) << 12;`

`x5 = (this1 & 0x00f00000) << 4;`

`x6 = (this1 & 0x0f000000) >> 20;`

`x7 = (this1 & 0xf0000000) >> 28;`

`return (x5 | x4 | x03 | x6 | x7);` // 所占位没有交叉情况下的合并 → 用 '|' ZONE

node this1; ^{前一个元素} ^{遍历} ^{存插入信息首节点}

```
5. node *previous, *temp, *interim, *first = NULL;
while(fgets(this1.name, 14, indata) != NULL && (fscanf(indata, "%d%c%f%f%*c", &(this1.emp-no), &(this1.gender), &(this1.rate), &(this1.hr)) == 4))
{
    if(first == NULL) // the list is empty
    {
        first = (node *) malloc(sizeof(node));
        if(first == NULL)
        {
            printf("malloc fails\n");
            return 0;
        }
        // 每次malloc完了别忘了判断一下内存是否申请成功
        *first = this1;
        first->next = NULL;
    }
    else // the list is not empty
    {
        interim = (node *) malloc(sizeof(node));
        if(interim == NULL)
        {
            printf("malloc fails\n");
            return 0;
        }
        else
        {
            *interim = this1;
            temp = first;
            while((temp->next != NULL) && (temp->emp-no < interim->emp-no))
            {
                previous = temp;
                temp = temp->next;
            }
            if(temp == first) // first node
            {
                interim->next = first;
                first = interim;
            }
        }
    }
}
```



```
else if ((temp->next == NULL) && (temp->emp-no < interim->emp-no))
{
```

```
temp->next = interim;
```

```
interim->next = NULL;
```

```
}
```

```
else // in between
```

```
}
```

```
previous->next = interim;
```

```
interim->next = temp;
```

```
}
```

```
{ // interim not equal to NULL
```

```
{ // the list is not empty
```

```
{ // while
```

相当于例教第3个元素的首地址

Tutorial 3

1. 从EOF开始
SEEK-END的光标在这儿
indata: A B C D E F G H
fseek(indata, (long)-3 * sizeof(char), SEEK_END);
fread(&ch, sizeof(char), 1, indata);

★ 2. fread 每成功读取一次, 会把文件指针向后移动读取的字节数
首地址位置给我画清楚
记得 fread 后光标会向后移动 fwrite 也会

3. w+b 只在打开文件的那一刻清空文件, 后续再 fread/fseek/fwrite 不会再次清空
%[^\n] 读取一整行字符串, 直到遇到 '\n' 为止 (但不包含 \n)

gets() 只能从 stdin 读取数据, 且会一直读取 input 直到遇到 \n 才停止 eg. char s[16];
gets(s);

4. int *num, *this1;

```
int sum;
```

```
num = (int *) malloc(2 * sizeof(int));
```

```
num[0] = num[1] = 10;
```

```
this1 = &num[0]; ★
```

```
realloc(num, 3 * sizeof(int));
```

```
* (this1 + 2) = 10;
```

```
sum = num[0] + num[1] + num[2];
```

eg. int a; 也是在申请内存, 但这是编译器在栈上自

⇒ sum may contain the wrong result 动帮你做的

because the returned new address of realloc function may not be the same as the previous address of num. It gets into problem if num is relocated by the realloc function.

若 realloc == NULL 原内存还在

但大小不变

写 num[2] = 越界 = 未定义行为

虽然你手里还存着那个地址, 但那块内存已经不属于你了: 不能再往里面读马
地址还在: 指针变量里的数值没变
使用权没了: 再访问就是未定义行为

CP4

1. 记得 `fread()`, `fwrite()` 后光标会向后移动

2. $x^x = 0$ $x^0 = x$

3. `char a[6][9]`

① `a` 是一个 array, 长度是 6; `a` 的每个元素又是一个 `char[9]`

• `a` 的类型是 `char[6][9]`

• `a[0]` 的类型是 `char[9]`

• `a[0][0]` 的类型是 `char`

② 怎么看多维数组的"元素"? $\Rightarrow$ 去掉最左边那一维, 剩的就是元素类型

eg. `char a[6][9]` 去掉最左边 `[6]`, 剩下 `char[9]`, 即 `a` 的元素类型

`int b[4][6][8]`

`b` 的类型 `int[4][6][8]`

元素类型 `int[6][8]`

`b[0]` `int[6][8]`

`int[8]`

`b[0][0]` `int[8]`

`int`

`char p[3][6][9]`

`b[0][0][0]` `int`

$\Rightarrow$ 注意 `char (*p)[6][9] = calloc(3, sizeof(*p));`

$\ast p$ 是 `char[9]`

③ 连续内存版:

`char (*p)[9] = (char (*)[9]) calloc(6, sizeof(*p));`

`p` 是一个指针, 指向 `char[9]` (长度为 9 的 `char` 数组) 一般是元素类型

④ 数组退化: 数组在大多数表达式中, 会自动转换成"指向它第一个元素的指针"

eg. `a[6][9]` 第一个元素是 `a[0]` 是 `char[9]`

指向第一个元素是 `char(*)[9]` $\therefore a$ 是 `char(*)[9]`

表达式

原始类型

退化后

`a`

`char[6][9]`

`char(*)[9]`

`a[i]`

`char[9]`

`char*`

`a[i][j]`

A

`char`

B

什么时候用"退化后的形式", 什么时候用"原始形式"

A. 用指针接: eg. `char (*p)[9] = a;`

B. 定义变量: `char[6][9]` `char row[9];`

⑤ 非连续内存版, 行 & 行不连续

`char **p = calloc(6, sizeof(char*));`

`for(--) p[i] = calloc(9, sizeof(char));`

Δ : `char **p = calloc(6, sizeof(char[9]));` `calloc(n, sizeof(*p))` 其中的

$\rightarrow$ 看类型时 / 元素类型时 $\ast p$ 必须和 `p` 指向的类型完全一致

$\ast$ `sizeof` 时

动态分配要匹配真实结构时

`calloc(6, sizeof(char[9]))`

$\checkmark$ `[char[9]][char[9]]`

$\times$ `[char*][char*]` ... 不能想退化